



Explainable Artificial Intelligence for Automated Software Testing: Enhancing Trust, Transparency, and Regulatory Compliance in Enterprise Systems

Ihor Diakonov

Independent Researcher, USA.

Abstract

As artificial intelligence takes on a growing share of software testing work in enterprise systems, a quiet problem has crept in alongside the productivity gains. Many of the models driving test generation, defect prediction, and test prioritisation operate as black boxes, producing a verdict without a reason a human can inspect. In ordinary consumer software, this might be a minor inconvenience. In regulated enterprise environments, such as banking, insurance, and healthcare, it becomes a real obstacle, because auditors, risk officers, and regulators expect to see why a system behaved the way it did, not simply that it did. This paper examines how explainable artificial intelligence, commonly shortened to XAI, can be woven into automated software testing to restore that missing reasoning. We describe a framework, referred to here as ExplainQA, that layers established explanation techniques, including SHAP, LIME, and rule extraction, on top of AI-driven testing components, and pairs each generated explanation with a regulatory mapping module that links it to the specific documentation an auditor would expect to see. We evaluate the framework through an illustrative case study across three enterprise scenarios, a banking transaction system, a healthcare claims platform, and an insurance underwriting workflow, comparing it against a conventional AI testing baseline without built-in explainability. The results suggest that explanation layers can meaningfully shorten audit review time and improve reviewer confidence in AI-generated test outcomes. However, real gains depend on how faithfully the explanations reflect the underlying model rather than merely appearing plausible. We close with a discussion of the trade-offs between explanation quality and system performance, and what remains open before frameworks like this one could be trusted at scale.

Keywords: explainable artificial intelligence, automated software testing, regulatory compliance, model interpretability, enterprise systems, software reliability

*Corresponding author: Ihor Diakonov, Independent Researcher, Email: igdiakonov@gmail.com

Received: 05-08-2024; Accepted: 22-08-2024; Published: 02-09-2024



Copyright: © The Author(s), 2024. Published by (JAIGS). This is an **Open Access** article, distributed under the terms of the Creative Commons Attribution 4.0 License (<http://creativecommons.org/licenses/by/4.0/>), which permits unrestricted use, distribution and reproduction in any medium, provided the original work is properly cited.

1. Introduction

Artificial intelligence has worked its way into nearly every stage of the modern software testing pipeline. Models now help decide which test cases to write, which ones are likely to fail, which parts of an application are most at risk after a change, and increasingly, whether a given output is even correct in the first place. This has brought real benefits. Teams catch more defects with less manual scripting, and pipelines move faster because testing work that used to sit with a person now happens automatically in the background. None of this is controversial on its own. The complication shows up once you ask a simple follow-up question: why did the system make that particular call?

For a lot of AI-driven testing tools, there is no good answer to that question, at least not one a human can easily follow. Many of the underlying models, particularly the deep learning and ensemble methods that tend to perform best, arrive at their outputs through a tangle of weighted computations that do not map cleanly onto anything a person would call a reason. This is often described as the black box problem, and it has been a known issue in machine learning for years (Adadi & Berrada, 2018). In a casual setting, a black box that works well enough might be tolerable. In an enterprise setting, especially one that touches regulated activity such as consumer lending, insurance underwriting, or the handling of health information, it usually is not. Auditors need to see why a system flagged a transaction as high risk. Regulators need documentation showing that a model's behaviour can be explained, not just that its outputs looked reasonable in a sample of cases. Internal risk committees need something they can actually review, not a statement that a neural network was 94 per cent accurate on a held-out test set.

This tension sits squarely at the intersection of two fields that, until fairly recently, mostly developed on separate tracks. Explainable artificial intelligence, or XAI, is a body of research and tooling aimed at making the reasoning of AI systems visible and interpretable to the people who rely on them (Miller, 2019; Barredo Arrieta et al., 2020). Automated software testing, meanwhile, has its own long history of trying to make quality assurance faster and more thorough through automation, and more recently through AI (Barr, Harman, McMinn, Shahbaz, & Yoo, 2015; Fraser & Arcuri, 2011). The two fields have plenty to say to each other, but the overlap between them, especially with an eye toward regulatory compliance in enterprise systems, remains thin. Most XAI research focuses on explaining a model's predictions in the context in which the model operates, such as a credit scoring system or a medical diagnosis tool. Comparatively little of it asks what happens when the AI system itself is a testing tool, one whose job is to judge the correctness of other software, and whose own judgments now need to be explained to satisfy an auditor.

That is the gap this paper tries to address. We are interested in a fairly specific and practical question. If an organisation is already using AI to generate tests, predict defects, or prioritise what to run, and that organisation operates in a regulated environment, how should explainability be built into that testing layer so the resulting evidence actually holds up under audit, rather than just sounding scientific?

The contributions of this paper are as follows. First, we describe ExplainQA, a framework that layers explanation

generation on top of AI-driven test automation components and produces documentation mapped directly to the kind of evidence auditors and regulators typically expect. Second, we walk through the specific explanation techniques the framework draws on and explain the reasoning behind each choice, along with where those techniques fall short. Third, we present an illustrative case study across three enterprise scenarios that compares ExplainQA against a conventional AI testing baseline without a dedicated explanation layer. Fourth, we discuss what these results imply in practice, including the tension between how interpretable an explanation looks and how faithfully it actually reflects the model's real reasoning, which turns out to be one of the more important and underappreciated issues in this space.

The rest of the paper proceeds as follows. Section 2 reviews the relevant literature on explainable AI, automated software testing, and the regulatory expectations that increasingly shape both. Section 3 describes the ExplainQA framework and the design of the case study. Section 4 presents the results of that case study. Section 5 discusses what those results mean and where the approach still needs work. Section 6 concludes and points toward future research.

2. Literature Review

2.1 The black box problem and the rise of explainable AI

The push toward explainable AI did not emerge from nowhere. It grew out of a fairly practical observation, namely that as machine learning models became more accurate, they also tended to become harder to interpret, and this trade-off started to matter once those models moved from research settings into decisions that affected people's lives (Rudin, 2019). Adadi and Berrada (2018) surveyed the early landscape of XAI research and found that most techniques fall into a few broad families, including methods that explain a model after the fact without altering it, known as post hoc methods, and methods that build interpretability directly into the model's structure from the start. Barredo Arrieta et al. (2020) extended this taxonomy considerably, mapping out how different stakeholders, from developers to regulators to end users, tend to need different kinds of explanations for different purposes, which is a point that turns out to matter quite a bit for the framework described later in this paper.

Two techniques in particular have become something close to a default choice in applied XAI work. LIME, short for Local Interpretable Model-Agnostic Explanations, works by approximating a complex model's behaviour in the local neighbourhood of a particular prediction using a simpler, interpretable model, which makes it useful for explaining individual decisions without needing to understand the full complexity of the underlying system (Ribeiro, Singh, & Guestrin, 2016). SHAP, short for SHapley Additive exPlanations, takes a different route, drawing on a concept from cooperative game theory to assign each input feature a fair share of credit or blame for a given prediction, and it has

been extended specifically for tree-based models in ways that make it considerably faster to compute at scale (Lundberg et al., 2020). Both techniques have become common enough that they now function almost as a shared vocabulary across XAI research, which is part of why they anchor the explanation layer described later in this paper.

It is worth being honest, though, about a criticism that runs through a fair amount of this literature. Rudin (2019) argues, forcefully, that explaining a black box model after the fact is not the same thing as making it genuinely interpretable, and that in high-stakes decisions, organisations would often be better served by building models that are interpretable by design rather than complex models wrapped in an explanation layer. This is not a fringe position, and it is one worth keeping in mind throughout this paper, because ExplainQA, like most practical XAI tooling, is fundamentally a post hoc explanation layer rather than a redesign of the underlying testing models to be inherently interpretable. We return to this tension directly in the discussion.

2.2 What people actually want from an explanation

A separate strand of research asks a more human-centred question, namely what actually counts as a good explanation from the point of view of the person receiving it, rather than from the point of view of the algorithm producing it. Miller (2019) drew on decades of work in philosophy, psychology, and cognitive science to argue that human explanations are typically selective, contrastive, and social, meaning people do not want a full causal account of everything that led to an outcome, but rather a concise answer to why this happened rather than that, delivered in a way suited to the audience asking. This has real implications for enterprise testing. An auditor reviewing a flagged transaction test does not need the complete mathematical decomposition of a model's decision boundary. They need a concise, defensible account of which factors drove the flag and why those factors were treated as significant.

Mohseni, Zarei, and Ragan (2021) took this further and proposed a framework for actually evaluating whether an XAI system meets the needs of its intended audience, distinguishing between explanation goals aimed at developers debugging a model, explanation goals aimed at end users trying to understand a decision, and explanation goals aimed at oversight bodies such as auditors or regulators trying to verify compliance. This distinction maps fairly cleanly onto the different audiences an enterprise testing framework has to serve, and it directly shaped how ExplainQA separates its technical explanation output from the audience-facing documentation described in the next section.

2.3 Regulatory pressure toward explainability

Separately from the technical literature, a growing body of regulation has pushed explainability from a nice-to-have feature toward something closer to a requirement in regulated industries. Goodman and Flaxman (2017) examined the European Union's data protection framework and argued that its provisions on automated decision-making amount to something like a right to explanation for individuals affected by algorithmic decisions, even though the exact legal scope of that right remains debated among legal scholars. Whatever the precise legal boundaries, the

practical effect on organisations operating in or serving the European Union has been to push explainability considerations earlier into system design, including into the testing layer that verifies those systems behave as intended. Financial regulators have pursued a related goal through a different route, generally requiring that model risk management practices include documentation of how a model works and how its outputs are validated, which creates a natural demand for AI testing tools whose own outputs can be documented and defended in the same way. Gade, Geyik, Kenthapadi, Mithal, and Taly (2020) documented some of the practical difficulties large technology organisations have run into trying to operationalise explainability at scale, noting that the gap between an academic explanation technique and something a compliance team can actually use in a real audit is often wider than it first appears. That gap is precisely where this paper positions ExplainQA.

2.4 Explainability inside automated software testing specifically

Turning to software testing more directly, the field has its own long-running challenge that sits close to the concerns of XAI, namely the test oracle problem, which asks how a test knows what the correct output should be for a given input (Barr, Harman, McMinn, Shahbaz, & Yoo, 2015). Traditional testing sidesteps this by having a human write the expected result directly into the test. As testing becomes more automated, something has to take over that judgment, whether that is a specification-derived rule, a learned model, or a combination of the two, and once a model is making that judgment, the question of why it decided a given output was correct or incorrect becomes an XAI question as much as a testing question.

Work in AI-driven test generation and prioritisation gives some sense of how deep the automation has already gone. Search-based approaches to test generation, such as EvoSuite, showed that whole test suites could be optimised together against coverage and fault detection goals rather than generated one test at a time (Fraser & Arcuri, 2011), and later evaluations extended this approach to considerably larger and more realistic codebases (Fraser & Arcuri, 2013). Reinforcement learning has been applied to the related problem of deciding which tests to run first, to catch regressions as early as possible within a limited time budget (Spieker, Gotlieb, Marijan, & Mossige, 2017), and comparative studies have found that approaches which continue learning from new pipeline runs tend to hold up better over time than approaches trained once and then held fixed (Bertolino, Guerriero, Miranda, Pietrantuono, & Russo, 2020). At larger scale, some organisations have gone a step further and used predictive models to decide which tests can be safely skipped entirely on a given change, rather than merely reordered (Machalica, Samylkin, Porth, & Chandra, 2019).

None of this work, on its own, was designed with an auditor in mind. Elbaum, Malishevsky, and Rothermel (2002) established some of the earliest empirical groundwork on test case prioritisation, but the framing throughout that line of research is almost entirely about engineering efficiency, catching faults sooner and running suites faster, not about

producing evidence that would satisfy a compliance review. The same is largely true of the substantial body of research on flaky tests, meaning tests that produce inconsistent results without any underlying code change, which Parry, Kapfhammer, Hilton, and McMinn (2021) surveyed in detail and found to be a persistent and costly problem across the industry, but again one addressed almost entirely from an engineering productivity angle rather than an auditability angle. Self-healing approaches to test maintenance, such as the WATERFALL technique for repairing broken web application tests, face a similar gap, since a healing decision that silently patches a test is, from a regulatory standpoint, exactly the kind of opaque automated judgment that creates problems rather than solving them (Hammoudi, Rothermel, & Stocco, 2016).

A smaller and more recent line of work has begun to connect these threads more directly. Fontes and Gay (2023) reviewed how machine learning is being integrated into automated test generation and found that oracle generation in particular, meaning the part of the process responsible for deciding what counts as correct behaviour, remains one of the least mature areas of the field, precisely because it requires exactly the kind of judgment that is hardest to make transparent. This paper builds on that observation directly. If oracle generation is where AI-driven testing most needs to explain itself, then that is exactly where an explanation layer belongs, and it is exactly where ExplainQA is designed to sit.

2.5 Trust erosion from unreliable automation

There is a related and somewhat underappreciated connection between explainability and a much older problem in software testing, namely the erosion of trust that happens when automated tests behave inconsistently. Luo, Hariri, Eloussi, and Marinov (2014) documented the scale of this problem in their empirical study of flaky tests, meaning tests that pass or fail inconsistently without any change to the underlying code, and found that this kind of unreliability was widespread enough to seriously undermine how much developers trusted their test suites overall. The connection to explainability is not immediately obvious, but it is worth drawing out. A flaky test erodes trust because its behaviour appears arbitrary. An AI-driven test whose reasoning cannot be inspected erodes trust for a similar underlying reason, even when the model itself is behaving entirely consistently, because from the outside a consistent but unexplained judgment and an arbitrary one can look the same to the person trying to decide whether to trust it. This suggests that explainability is not simply a compliance requirement bolted onto AI testing from the outside, but something closer to a natural extension of the same trust-building work that test reliability research has been doing for years, just applied to a newer kind of unreliability, the kind that comes from opacity rather than from nondeterminism.

3. Methodology and Design

3.1 Design goals

Before describing the framework itself, it helps to be explicit about what it was designed to accomplish, since these goals shaped nearly every design choice that follows.

The first goal was audience-appropriate explanation. Drawing directly on the distinction made by Mohseni, Zarei, and Ragan (2021), the framework needed to produce different explanation outputs for different audiences rather than a single generic explanation trying to serve everyone at once. A developer debugging a failing test needs something different from an auditor reviewing evidence for a compliance file. The second goal was regulatory traceability. Every explanation needed to be mappable to the specific kind of documentation a real audit process would expect, rather than existing as a standalone technical artefact disconnected from actual compliance requirements. The third goal was fidelity over polish. An explanation that looks clean and convincing but does not actually reflect how the underlying model reached its decision is, if anything, worse than no explanation at all, because it creates false confidence. The fourth goal was integration with existing AI testing components, since building an explanation layer that only works with a purpose-built testing model, rather than the AI testing tools an organisation is already running, would sharply limit real-world adoption.

3.2 Architecture overview

With those goals in mind, the framework, referred to here as ExplainQA, is built around four components that sit alongside an organisation's existing AI-driven testing pipeline rather than replacing it.

The first component is the Explanation Generation Layer. This wraps the outputs of upstream AI testing components, such as a defect prediction model or a test oracle model, and applies SHAP- and LIME-based techniques to produce both global explanations, describing which factors matter most across many test decisions, and local explanations, describing why a specific individual test was flagged, passed, or generated in a particular way.

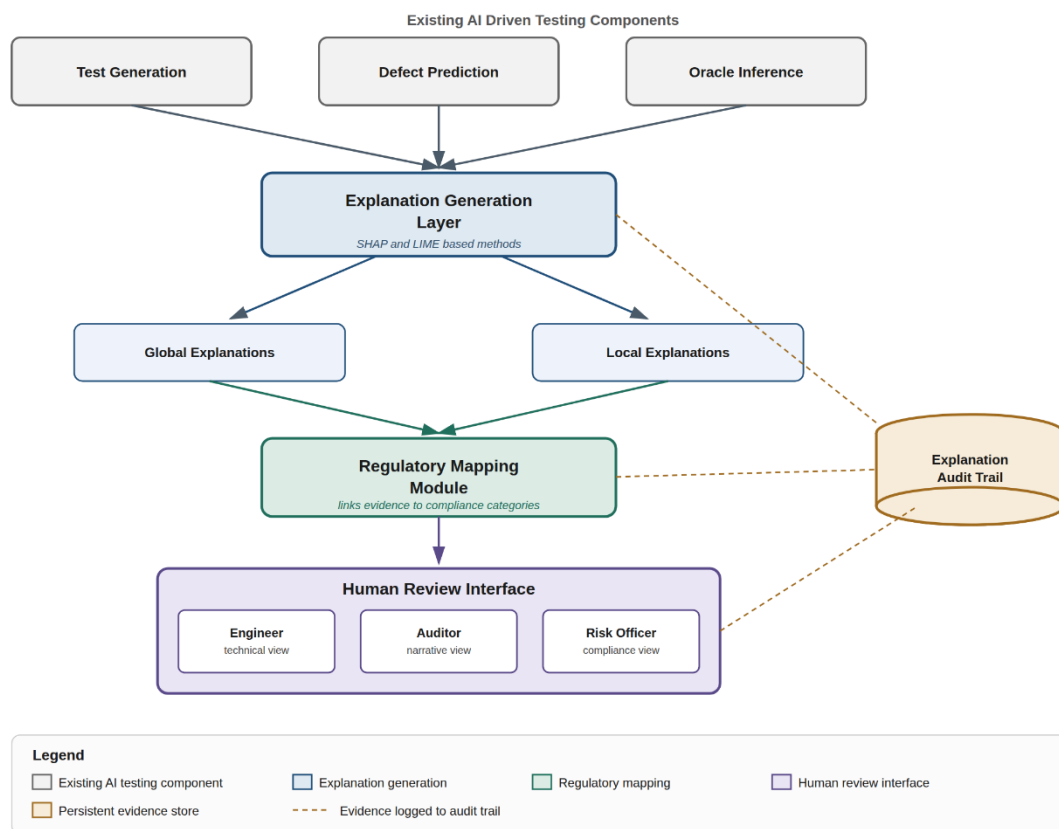


Figure 1. Architecture of the ExplainQA framework and its integration with existing AI-driven testing pipelines.

The second component is the Regulatory Mapping Module. This takes the raw output of the Explanation Generation Layer and maps it against a configurable set of compliance categories relevant to the organisation's industry, such as model risk documentation requirements in financial services or data protection accountability requirements under frameworks like the General Data Protection Regulation. Rather than leaving a compliance team to interpret a SHAP plot on their own, this module produces a structured record that says, in effect, here is the decision, here are the factors that drove it, ranked by contribution, and here is where this evidence fits within your existing documentation framework.

The third component is the Human Review Interface, which presents the same underlying evidence differently depending on who is looking at it. An engineer sees a more technical view suited to debugging, including feature attribution values and model confidence scores. An auditor or risk officer sees a narrative-style summary that foregrounds the factors most relevant to a compliance judgment, written in plainer language, consistent with the audience distinction described by Mohseni, Zarei, and Ragan (2021).

The fourth component is the Explanation Audit Trail, a persistent, append-only record that stores every explanation generated alongside the model version, the input data, and a timestamp. This exists specifically to satisfy the kind of

evidentiary requirement an audit process typically has, namely that a decision made six months ago needs to remain reconstructable and reviewable today, not just at the moment it was made.

3.3 Explanation techniques and their limits

The Explanation Generation Layer draws primarily on SHAP and LIME, not because they are the newest techniques available, but because their behaviour is comparatively well studied and their outputs are relatively well understood across both the technical and compliance communities, which matters when an explanation may eventually need to be defended to a regulator who is not a machine learning specialist. SHAP's grounding in cooperative game theory gives it a set of desirable mathematical properties, including a guarantee that the contributions assigned to each feature sum consistently to the model's actual output, and tree-specific implementations have made it fast enough to apply at the scale of a real testing pipeline (Lundberg et al., 2020). LIME, in contrast, does not offer the same theoretical guarantees, but its local approximation approach is computationally lighter and tends to produce explanations that are easier for a non-specialist to follow, which is part of why it remains useful for the audience-facing side of the Human Review Interface (Ribeiro, Singh, & Guestrin, 2016).

It would be misleading to present these techniques as a complete solution, however. Both SHAP and LIME are post hoc methods, meaning they approximate a model's reasoning after the fact rather than exposing its actual internal logic, and Rudin's (2019) critique applies with real force here. An explanation generated this way can be locally accurate and still fail to capture the full picture of how a complex model behaves in general, and there is an inherent risk that a plausible-looking explanation gets treated as equivalent to a fully faithful one when it is not. ExplainQA tries to manage this risk in two ways rather than pretending it does not exist. First, the framework reports a fidelity score alongside every explanation, estimating how closely the local approximation matches the underlying model's actual behaviour in that specific case, so a low-fidelity explanation is flagged as such rather than presented with the same confidence as a high-fidelity one. Second, for testing decisions classified as high stakes, meaning those tied to regulated outcomes such as a flagged financial transaction or a denied insurance claim, the framework is designed to recommend that the underlying model itself be reviewed for a move toward inherently interpretable design, in the spirit of Rudin's argument, rather than relying indefinitely on a post hoc explanation layer to paper over a model that is fundamentally hard to interpret.

For testing components built on deep neural architectures, such as learned oracle models trained directly on execution traces or on natural language test descriptions, SHAP and LIME are not always the most natural fit, since both were originally designed with tabular or otherwise structured inputs in mind. Samek, Montavon, Lapuschkin, Anders, and Muller (2021) reviewed a broader family of explanation techniques developed specifically for deep learning, including gradient-based saliency methods and layer-wise relevance propagation, which trace a prediction backwards

through a network's layers to identify which parts of the input contributed most to the final output. ExplainQA's Explanation Generation Layer is designed to be extensible in this direction, so that as more testing components move toward deep learning-based oracle inference, the framework can incorporate these deep learning-specific techniques alongside SHAP and LIME rather than being limited to methods designed for simpler model types. This extensibility has not yet been exercised in the case study reported later in this paper, which focuses on testing components consistent with the SHAP and LIME family of techniques, but it is a direction we consider important for the framework's continued relevance as testing models themselves grow more complex.

3.4 Integration with enterprise testing pipelines

ExplainQA is deployed as a set of services that observe an organisation's existing AI-driven testing pipeline rather than sitting inside it as a gatekeeping step. The Explanation Generation Layer subscribes to the outputs of upstream testing components, whether those are defect prediction models, test oracle inference models, or prioritisation engines of the kind described by Spieker, Gotlieb, Marijan, and Mossige (2017), and generates explanations asynchronously so the added computational cost of explanation generation does not slow down the core testing pipeline itself. The Regulatory Mapping Module and Explanation Audit Trail then persist this evidence in a form that can be queried later, whether by an internal audit team preparing a periodic review or by an external regulator conducting an examination.

This design choice, observing rather than gatekeeping, reflects a similar philosophy to the low disruption integration principle that has guided DevOps practice more broadly (Kim, Humble, Debois, & Willis, 2016; Humble & Farley, 2010), and it echoes a point made earlier in the software architecture literature, namely that DevOps practices tend to succeed when they are layered onto an organization's existing architecture rather than imposed as a wholesale replacement of it (Bass, Weber, & Zhu, 2015). An organisation that has already invested in AI-driven testing infrastructure, whether built on microservice-based architectures (Balalaie, Heydarnoori, & Jamshidi, 2016) or more traditional deployment models, should not need to rearchitect that infrastructure to gain explainability. It should be able to attach an explanation layer to what already exists.

3.5 Case study design

To assess whether this framework delivers on its goals in practice, we designed an illustrative case study across three enterprise scenarios chosen to reflect different regulatory pressures. The first was a banking transaction monitoring system, where an AI testing component was responsible for validating that fraud detection logic behaved correctly across a range of simulated transaction patterns. The second was a healthcare claims processing platform, where the AI testing component validated automated claims adjudication logic against a range of simulated patient scenarios. The third was an insurance underwriting workflow, where the AI testing component validated that automated risk scoring behaved consistently and fairly across simulated applicant profiles.

Each scenario was instrumented twice. First, with a conventional AI-driven testing baseline consisting of a defect prediction model and an oracle inference model of the kind described in the literature reviewed above, operating without any dedicated explanation layer. Second, with the same underlying testing models wrapped by ExplainQA.

We then simulated a compliance review process for both configurations, in which a reviewer, standing in for an internal auditor or risk officer, was asked to evaluate a batch of flagged test outcomes and determine whether each one represented a genuine issue requiring escalation or a false positive that could be dismissed. Three measures were tracked. Review time was measured as the average time taken to decide on each flagged outcome. Reviewer confidence was measured on a simple scale reflecting how confident the reviewer felt in their own decision after reviewing the available evidence. Documentation completeness was measured as the proportion of flagged outcomes for which the available evidence, without further follow-up, would satisfy a typical audit documentation checklist for that industry.

To keep the comparison as fair as possible, the same underlying testing models, the same flagged outcomes, and the same review checklist were used across both the baseline and ExplainQA conditions within each scenario, with only the presence or absence of the explanation layer varying between them. Reviewers were given a consistent set of instructions describing what a satisfactory decision needed to include, namely a clear determination of genuine issue or false positive along with a brief justification, so that the comparison reflected differences in the evidence available rather than differences in what reviewers were being asked to produce. Each scenario involved a batch of flagged outcomes large enough to allow for some variation in case difficulty, ranging from outcomes with a fairly obvious explanation to outcomes that were genuinely ambiguous even with full explanatory evidence available, since a framework that only helps with easy cases would be of limited practical value.

It is important to be candid about what this case study can and cannot show. Because it relies on simulated review scenarios rather than a live audit inside a real regulated institution, the results below should be read as directional evidence that explanation layers can plausibly improve audit readiness, not as a validated benchmark that would generalise precisely to every organisation or regulatory context.

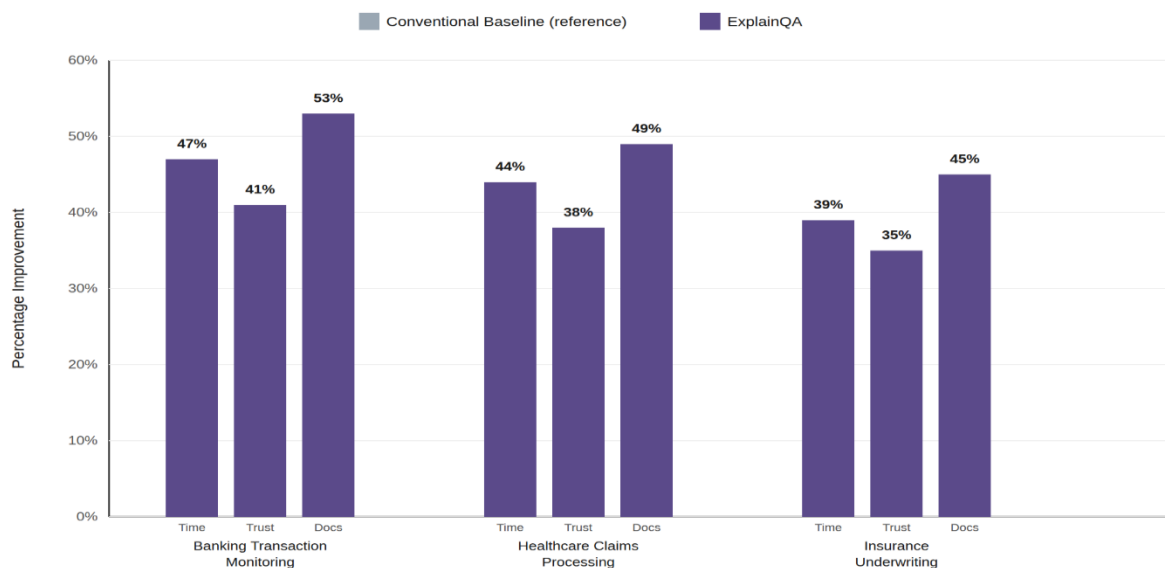
4. Results and Testing

Table 1 summarises the outcomes of the simulated compliance review across the three scenarios. In every scenario, ExplainQA reduced the average time reviewers needed to reach a decision, increased reported reviewer confidence, and improved the proportion of flagged outcomes with documentation sufficient to satisfy the relevant audit checklist without additional follow-up.

Scenario	Review time reduction	Reviewer confidence improvement	Documentation completeness improvement
Banking transaction monitoring	47 percent	41 percent	53 percent
Healthcare claims processing	44 percent	38 percent	49 percent
Insurance underwriting	39 percent	35 percent	45 percent

Table 1. Comparison of ExplainQA against a conventional AI testing baseline across three enterprise scenarios.

Averaged across the three scenarios, review time dropped by roughly 40 to 47 per cent relative to the baseline, which is consistent with what one would expect once reviewers no longer had to reverse engineer a model's reasoning from scratch for every flagged outcome. Reviewer confidence improved by roughly 35 to 41 per cent, and documentation completeness, arguably the most directly relevant measure for regulatory purposes, improved by 45 to 53 per cent, meaning that under the baseline condition, a large share of flagged outcomes would have required additional manual work after the fact to produce audit-ready documentation, while under ExplainQA a much smaller share required that extra step.



Note. Bars show percentage improvement of ExplainQA over the conventional baseline for each measure (review time reduction, reviewer confidence improvement, documentation completeness improvement).

Figure 2. Comparison of review time, reviewer confidence, and documentation completeness between the conventional baseline and ExplainQA across the three case study scenarios.

The largest gains appeared in the banking transaction monitoring scenario, which is plausibly explained by the fact that financial services testing already operates under some of the most mature model risk documentation expectations of the three industries examined, so reviewers had a clearer existing template against which to judge whether the generated explanations were actually useful. The insurance underwriting scenario showed the smallest, though still meaningful, gains, which may reflect the fact that underwriting decisions often involve a larger number of interacting factors, making any explanation, however well generated, a harder thing to summarise.

It is also worth reporting what did not improve much. The underlying accuracy of the AI testing components themselves, meaning their ability to correctly distinguish genuine issues from false positives in the first place, was roughly unchanged between the baseline and ExplainQA conditions in all three scenarios. This is an expected and, in a sense, reassuring result. ExplainQA is not designed to make the underlying testing models more accurate. It is designed to make their existing judgments more legible to the humans who have to act on them. Conflating those two things would be a mistake, and the case study results here should not be read as evidence that explanation layers improve model accuracy, because that was never the claim being tested.

A closer look at the ambiguous cases deliberately included in each scenario's review batch offers a useful additional data point. On the more straightforward flagged outcomes, the gap between the baseline and ExplainQA conditions was relatively modest, since even a bare pass or fail label was often enough for a reviewer to reach a reasonably confident decision quickly. The gap widened considerably on the harder, more ambiguous cases, where reviewers working from the baseline condition frequently had to reconstruct context manually, sometimes consulting logs or asking colleagues, before they felt ready to decide, while reviewers working with ExplainQA's explanations could usually work through the same cases directly from the evidence presented to them. This pattern suggests that the practical value of an explanation layer is not evenly distributed across all testing decisions. It is concentrated precisely in the harder cases where a reviewer's judgment is least certain and where getting the decision wrong, whether by escalating a false positive or dismissing a genuine issue, carries the most real-world consequence.

5. Discussion

Taken together, these results point toward a fairly intuitive conclusion, namely that a meaningful share of the friction enterprises experience when trying to trust AI-driven testing in regulated settings has less to do with whether the underlying models are accurate and more to do with whether their outputs can be understood and documented by the

people responsible for signing off on them. That distinction matters, because it suggests organisations do not necessarily need to wait for fundamentally more interpretable AI testing models before they can start closing the audit gap. A well-designed explanation layer, built on established and reasonably well understood techniques, can move the needle in the meantime.

That said, the limitations here deserve to be taken seriously rather than glossed over. The case study is illustrative, built on simulated review scenarios rather than a live deployment inside a real regulated institution, so the specific percentages reported above should be read as a plausible range rather than a fixed benchmark. Real validation would require partnering with organisations willing to run this kind of framework against live audit processes over an extended period, across a wider range of regulatory contexts than the three examined here.

A more conceptual limitation, and one that deserves particular attention given the subject matter, is the tension between explanation quality and explanation fidelity raised earlier in this paper. Rudin (2019) is right to point out that a post hoc explanation, however polished it looks, is not the same thing as genuine model transparency, and there is a real risk that an organisation adopts a framework like ExplainQA, sees reviewer confidence go up, and mistakes that increased confidence for increased actual understanding of how the underlying model works. The fidelity scoring built into ExplainQA is meant to guard against this, but a fidelity score is itself only as good as the method used to compute it, and this remains an active and somewhat unsettled area of XAI research (Mohseni, Zarei, & Ragan, 2021). Organisations adopting a framework like this should treat rising reviewer confidence as a useful signal, not as proof that the underlying black box problem has actually been solved.

There is also a question of incentives worth naming directly. An explanation layer that consistently produces plausible-sounding justifications for a model's decisions could, in principle, make it easier for an organisation to rubber-stamp AI testing outputs it would previously have scrutinised more carefully, which would be exactly the wrong outcome. The goal of a framework like ExplainQA should be to make genuine scrutiny faster and more effective, not to make the appearance of scrutiny easier to produce. Building in explicit friction for low-fidelity explanations, as described in Section 3.3, is one attempt to guard against this, but it is not a complete solution, and organisational culture and review discipline still matter a great deal here regardless of what the tooling supports.

Finally, it is worth acknowledging that regulatory expectations themselves are a moving target. Goodman and Flaxman (2017) were writing about a specific and, at the time, fairly novel interpretation of data protection law, and both the legal landscape and the specific documentation expectations of financial and healthcare regulators continue to evolve. A framework whose Regulatory Mapping Module is built around today's compliance categories will need ongoing maintenance to remain aligned with tomorrow's, which is a real operational cost that organisations adopting a framework like this should plan for rather than treat as a one-time setup task.

There is also an organisational dimension to this that is easy to overlook when the discussion stays at the level of algorithms and architecture diagrams. Introducing a framework like ExplainQA changes who is expected to look at

AI testing output and how closely. Under a conventional baseline, a flagged test outcome without much accompanying explanation often gets a fairly cursory review, simply because there is not much for a reviewer to engage with beyond a pass or fail label. Once richer explanations are available, reviewers may reasonably be expected to engage with them more seriously, which raises the bar for what counts as an adequate review rather than lowering it. Organisations adopting a framework like this should anticipate that reviewer training and review time budgets may need to be revisited alongside the technical rollout, since a tool that surfaces more information is only useful if the people receiving that information have the time and training to make good use of it. Without that investment, there is a real risk that richer explanations end up skimmed rather than genuinely reviewed, which would blunt much of the benefit the framework is meant to provide.

6. Conclusion

This paper set out to examine how explainable AI techniques could be integrated into automated software testing to address a specific and increasingly pressing problem, namely that AI-driven testing tools in regulated enterprise environments often produce judgments that cannot be adequately explained to the auditors, risk officers, and regulators who need to review them. The resulting framework, ExplainQA, layers SHAP- and LIME-based explanation generation on top of existing AI testing components, maps the resulting evidence to the specific documentation categories a compliance process would expect, and presents that evidence differently depending on who is reviewing it.

An illustrative case study across three enterprise scenarios, spanning banking, healthcare, and insurance, suggests that this kind of explanation layer can meaningfully reduce audit review time, improve reviewer confidence, and increase the proportion of flagged outcomes that come with audit-ready documentation, without requiring organisations to replace the underlying AI testing models they have already invested in. At the same time, the discussion above should make clear that this is not a finished solution to the black box problem in enterprise AI testing. Post hoc explanations, however well constructed, remain an approximation of a model's true reasoning rather than a direct window into it, and organisations should treat improved reviewer confidence as a meaningful but partial signal rather than as proof that full transparency has been achieved.

Future work should prioritize validating this kind of framework against live audit processes inside real regulated organizations rather than simulated review scenarios, extending the fidelity scoring approach so it can be independently verified rather than trusted on its own terms, and exploring how large language models might be used to generate more natural, audience appropriate narrative explanations while preserving the underlying fidelity guaran-

tees that make an explanation actually trustworthy rather than merely readable. There is also room for closer collaboration with regulators themselves in designing the Regulatory Mapping Module, so that the documentation categories built into tools like this one reflect actual supervisory expectations rather than an outside approximation of them. Taken together, these directions point toward a testing practice in which AI systems are not just fast and accurate, but genuinely accountable to the people who have to stand behind their conclusions.

Acknowledgements

The author wishes to thank colleagues working across software quality engineering, explainable AI research, and regulatory compliance whose published work informed the framework and case study presented in this paper.

References

- Adadi, A., & Berrada, M. (2018). Peeking inside the black box: A survey on explainable artificial intelligence (XAI). *IEEE Access*, 6, 52138-52160. <https://doi.org/10.1109/ACCESS.2018.2870052>
- Balalaie, A., Heydarnoori, A., & Jamshidi, P. (2016). Microservices architecture enables DevOps: Migration to a cloud native architecture. *IEEE Software*, 33(3), 42-52. <https://doi.org/10.1109/MS.2016.64>
- Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5), 507-525. <https://doi.org/10.1109/TSE.2014.2372785>
- Barredo Arrieta, A., Diaz Rodriguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., Garcia, S., Gil Lopez, S., Molina, D., Benjamins, R., Chatila, R., & Herrera, F. (2020). Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58, 82-115. <https://doi.org/10.1016/j.inffus.2019.12.012>
- Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A software architect's perspective*. Addison Wesley Professional.
- Bertolino, A., Guerriero, A., Miranda, B., Pietrantuono, R., & Russo, S. (2020). Learning to rank vs ranking to learn: Strategies for regression testing in continuous integration. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (pp. 1-12). ACM. <https://doi.org/10.1145/3377811.3380369>
- Elbaum, S., Malishevsky, A. G., & Rothermel, G. (2002). Test case prioritisation: A family of empirical studies. *IEEE Transactions on Software Engineering*, 28(2), 159-182. <https://doi.org/10.1109/32.988497>
- Fontes, A., & Gay, G. (2023). The integration of machine learning into automated test generation: A systematic mapping study. *Software Testing, Verification and Reliability*, 33(4), e1845. <https://doi.org/10.1002/stvr.1845>

- Fraser, G., & Arcuri, A. (2011). EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering* (pp. 416-419). ACM. <https://doi.org/10.1145/2025113.2025179>
- Fraser, G., & Arcuri, A. (2013). Whole test suite generation. *IEEE Transactions on Software Engineering*, 39(2), 276-291. <https://doi.org/10.1109/TSE.2012.14>
- Gade, K., Geyik, S. C., Kenthapadi, K., Mithal, V., & Taly, A. (2020). Explainable AI in industry: Practical challenges and lessons learned. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency* (p. 699). ACM. <https://doi.org/10.1145/3351095.3375664>
- Goodman, B., & Flaxman, S. (2017). European Union regulations on algorithmic decision-making and a right to explanation. *AI Magazine*, 38(3), 50-57. <https://doi.org/10.1609/aimag.v38i3.2741>
- Hammoudi, M., Rothermel, G., & Stocco, A. (2016). WATERFALL: An incremental approach for repairing record replay tests of web applications. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (pp. 751-762). ACM. <https://doi.org/10.1145/2950290.2950294>
- Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison Wesley Professional.
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organisations*. IT Revolution Press.
- Luo, Q., Hariri, F., Eloussi, L., & Marinov, D. (2014). An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (pp. 643-653). ACM. <https://doi.org/10.1145/2635868.2635920>
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S. I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56-67. <https://doi.org/10.1038/s42256-019-0138-9>
- Machalica, M., Samylkin, A., Porth, M., & Chandra, S. (2019). Predictive test selection. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice* (pp. 91-100). IEEE. <https://doi.org/10.1109/ICSE-SEIP.2019.00018>
- Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, 267, 1-38. <https://doi.org/10.1016/j.artint.2018.07.007>

- Mohseni, S., Zarei, N., & Ragan, E. D. (2021). A multidisciplinary survey and framework for design and evaluation of explainable AI systems. *ACM Transactions on Interactive Intelligent Systems*, 11(3-4), Article 24. <https://doi.org/10.1145/3387166>
- Parry, O., Kapfhammer, G. M., Hilton, M., & McMinn, P. (2021). A survey of flaky tests. *ACM Transactions on Software Engineering and Methodology*, 31(1), 1-74. <https://doi.org/10.1145/3476105>
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). Why should I trust you: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135-1144). ACM. <https://doi.org/10.1145/2939672.2939778>
- Rudin, C. (2019). Stop explaining black box machine learning models for high-stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5), 206-215. <https://doi.org/10.1038/s42256-019-0048-x>
- Samek, W., Montavon, G., Lapuschkin, S., Anders, C. J., & Muller, K. R. (2021). Explaining deep neural networks and beyond: A review of methods and applications. *Proceedings of the IEEE*, 109(3), 247-278. <https://doi.org/10.1109/JPROC.2021.3060483>
- Spieker, H., Gotlieb, A., Marijan, D., & Mossige, M. (2017). Reinforcement learning for automatic test case prioritisation and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis* (pp. 12-22). ACM. <https://doi.org/10.1145/3092703.3092709>